



TECNOLÓGICO NACIONAL DE MEXICO

INSTITUTO TECNOLÓGICO DE CIUDAD JUÁREZ

DEPARTAMENTO DE SISTEMAS COMPUTACIONALES

NCIE

**Digitalización de Procesos Internos del Nodo:
Manual Técnico v1.0**

FECHA DE ENTREGA: 12 DE MARZO DE 2026

Gestor:

Edgar David Palacios Flores

Alumno:

Oscar Chavez Muñoz

Control de Versiones

Versión	Fecha	Autor	Descripción de Cambios
1.0	10 de Marzo, 2026	Oscar Chavez Muñoz	Emisión inicial del manual de arquitectura y especificaciones del sistema NCIE

****Documento Confidencial | Propiedad de NCIE | Versión 1.0****

Índice

1. Resumen Ejecutivo	4
2. Arquitectura del Sistema	4
2.1. Diagrama de Arquitectura	4
2.2. Componentes Arquitectónicos	5
2.3. Ecosistema Frontend	5
3. Ingeniería y Modelo de Datos (Esquemas Entidad-Relación)	6
3.1. Módulo de Gestión de Identidad (IAM)	6
3.2. Módulo Operativo - Core Académico	6
3.3. Módulo de Proyectos (Innovación / Startups)	6
3.4. Entidades Maestras (Catálogos)	7
4. Directrices Estándar de Mantenimiento	7
5. Procedimiento de Despliegue (v1.0)	7
6. Validación del Despliegue	14
6.1. Estado del servicio Apache	14
6.2. Accesibilidad local	14
6.3. Acceso externo	14
6.4. Verificación de logs	14
6.5. Validación funcional	15
7. Mantenimiento Operativo	15
8. Problemas Comunes y Soluciones	15
9. Plan de Rollback y Recuperación ante Desastres	17
9.1. Fallos durante el Despliegue Inicial (Rollback por instalación)	17
9.2. Protocolos de Actuación por Incidente (Casos Críticos)	17
10. Política de Respaldos Cíclicos (Backups Automáticos)	18
11. Buenas Prácticas de Producción	18

1. Resumen Ejecutivo

El presente Manual Técnico consolida la documentación integral sobre la arquitectura, diseño de componentes, modelo de datos relacional y procedimiento de despliegue para la plataforma NCIE (Nodo de Creatividad, Innovación y Emprendimiento). Está dirigido a mantenedores de software, ingenieros de DevOps y líderes técnicos encargados de la evolución, soporte, instalación y recuperación del código fuente en el servidor institucional. Esta documentación refleja la **Versión 1.0** del sistema, utilizando las herramientas nativas del repositorio (PHP, Laravel, MySQL local y comandos integrados), sentando las bases operativas previo a una futura estandarización arquitectónica institucional (Versión 2).

2. Arquitectura del Sistema

El sistema NCIE está edificado sobre el framework Laravel (Versión 10) bajo el paradigma de diseño MVC (Modelo-Vista-Controlador).

2.1. Diagrama de Arquitectura

Implementa una arquitectura en capas, separando la presentación, lógica de negocio y persistencia de datos.

- **Cliente:** Navegador web del usuario final que consume la aplicación mediante HTTP.
- **Servidor de Aplicaciones:** Laravel ejecutándose sobre PHP, encargado de la lógica de negocio.
- **Controladores:** Gestionan las solicitudes y coordinan la interacción entre vistas y modelos.
- **Modelos (Eloquent ORM):** Encapsulan la lógica de acceso a datos.
- **Base de Datos:** MySQL, responsable de la persistencia de la información.

Flujo de operación: Usuario → Navegador → Aplicación (Laravel) → Base de Datos

A continuación, se muestra el diagrama de arquitectura del sistema:

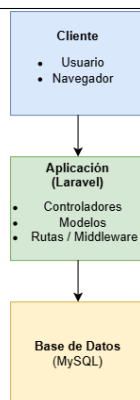


Figura 1: Diagrama de Arquitectura del Sistema NCIE

2.2. Componentes Arquitectónicos

- **Capa de Enrutamiento y Middleware:** Todas las peticiones HTTP son interceptadas por los archivos de la carpeta **routes/**. Los middlewares se encargan de validar la sesión y aplicar las políticas de Control de Acceso (ACL) utilizando **spatie/laravel-permission** antes de entregar el flujo a los Controladores.
- **Controladores (Capa Lógica):** Residen en **app/Http/Controllers/**. Contienen la lógica de negocio imperativa, mediando entre las interacciones del usuario y la persistencia de datos. Extraen parámetros de los objetos Request, invocan la capa de abstracción de datos e inyectan las respuestas hacia las vistas.
- **Modelos (Capa de Datos - ORM):** Ubicados en **app/Models/**. Implementan el patrón Active Record vía Eloquent. Gestionan de forma fluida el mapeo de tablas relacionales, ocultando consultas SQL complejas.
- **Vistas (Capa de Presentación):** Localizadas en **resources/views/**. Compuestas mediante el motor de plantillas Blade, permitiendo herencia de layouts integrados con directivas declarativas.

2.3. Ecosistema Frontend

La estilización de la capa de visualización emplea el preprocesador Sass encapsulando Bootstrap 5. El empaquetado y hot-reloading está orquestado altamente paralelo por Vite (**vite.config.js**), transpilando SCSS y Javascript/ESM estricto a los binarios optimizados en **public/build/**.

3. Ingeniería y Modelo de Datos (Esquemas Entidad-Relación)

La base de datos se rige por integridad referencial estricta. Las transacciones se concentran en cuatro pilares lógicos:

3.1. Módulo de Gestión de Identidad (IAM)

- **users** (Modelo **User**): Repositorio principal de autenticación. Entidades vinculadas unívocamente a los roles (Spatie ACL).
- **Tablas derivadas de Spatie: roles, permissions, model_has_roles**, conformando la topología RBAC (Role-Based Access Control).

3.2. Módulo Operativo - Core Académico

- **alumnos** (Modelo **Alumno**): Registro demográfico y operativo de los beneficiarios.
- **cursos** (Modelo **Curso**): Especificación de unidades de capacitación técnica.
- **alumno_curso** (Modelo **Inscripcion**): Conector cardinal **many-to-many** para historial de matriculación.
- **gestor_curso** (Modelo **GestorCurso**): Conector para la asignación de administradores/instructores de curso.
- **horarios** (Modelo **Horario**): Lógica del componente temporal de los cursos.

3.3. Módulo de Proyectos (Innovación / Startups)

- **proyectos** (Modelo **Proyecto**): Nomenclatura, categorización y estado del ciclo de vida de los proyectos productivos.
- **alumno_proyecto** (Modelo **AlumnoProyecto**): Mapeo de co-autores y desarrolladores.
- **gestor_proyecto** (Modelo **GestorProyecto**): Mapeo de supervisores, tutores o gestores líderes.
- **presupuestos** (Modelo **Presupuesto**): Módulo contable subyacente que rastrea partidas, gastos o apalancamientos por proyecto.

3.4. Entidades Maestras (Catálogos)

- **areas** (Modelo **Area**): Unidades lógicas departamentales o físicas del Nodo.
- **posts** (Modelo **Post**): Sistema base para avisería, notas de blog o bitácoras para los involucrados.

4. Directrices Estándar de Mantenimiento

Todo miembro del equipo de ingeniería debe adherirse a los siguientes estándares para el mantenimiento correctivo y evolutivo:

1. **Gestión de Cambios en la Base de Datos:** Jamás modificar la estructura de una tabla DDL mediante consolas SQL crudas. Todo cambio DEBE trazarse mediante una migración (`php artisan make:migration modifica_columna_x_en_tabla_y`).
2. **Validaciones de Requests:** Prohibido validar peticiones dentro del cuerpo metodológico grande del controlador. Mover cualquier lógica de validación voluminosa a Form Requests autónomos (`app/Http/Requests`).
3. **Optimización de Consultas N+1:** Toda iteración masiva inter-modelos en plantillas Blade exige Eager Loading en el Controlador usando la directiva `with()` para salvaguardar el rendimiento en memoria.
4. **Registro Log y Monitoreo:** Los eventos de fallo crítico están enlazados al sistema Logger nativo que vierte sobre `storage/logs/laravel.log`. La depuración exhaustiva debe consultar primeramente este fichero.

5. Procedimiento de Despliegue (v1.0)

Paso 1: Preparación del Entorno en Ubuntu

1. Acceder al servidor institucional vía SSH.
2. Actualizar el gestor de paquetes del sistema operativo:

```
sudo apt update && sudo apt upgrade -y
```
3. Instalar PHP 8.1, sus extensiones, Apache, MySQL Server, Node.js y npm:

```
#Instalacion del servidor web y base de datos  
sudo apt install -y apache2 mysql-server
```

```
#Instalacion de PHP y extensiones necesarias
sudo apt install -y php8.1 php8.1-cli php8.1-common php8.1-mysql php8.1-xml php8.1-
mbstring php8.1-curl php8.1-zip php8.1-bcmath php8.1-intl libapache2-mod-php

#Instalacion de Node.js y npm
sudo apt install -y nodejs npm

#Verificacion de versiones instaladas
php -v
node -v
npm -v
mysql --version
```

4. Instalar Composer globalmente y la herramienta Git:

```
sudo apt install -y git curl
curl -sS https://getcomposer.org/installer | php
sudo mv composer.phar /usr/local/bin/composer
```

Paso 2: Obtención del Proyecto

1. Navegar al directorio donde se alojará el proyecto:

```
sudo mkdir -p /var/www/ncieportal

cd /var/www/ncieportal
```

2. Clonación del repositorio mediante SSH:

Dado que el repositorio del proyecto es privado, se utilizará autenticación mediante claves SSH para garantizar acceso seguro y sin uso de credenciales manuales.

- a) Generación de clave SSH en el servidor:

```
ssh-keygen -t ed25519 -C "ncie@institucion.com"
```

Acepta los valores por defecto.

- b) Obtención de la clave pública:

```
cat ~/.ssh/id_ed25519.pub
```

- c) Registro de la clave en GitHub: La clave generada deberá registrarse en el repositorio de la plataforma GitHub, en la sección: Settings → SSH and GPG Keys → New SSH Key.
- d) Verificación de conexión: **ssh -T git@github.com** Se debe recibir un mensaje de autenticación exitosa.
- e) Permiso de carpetas: **sudo chown -R USER :USER /var/www/ncieportal**
- f) Clonación del repositorio: **git clone git@github.com:NCIE-2026/NCIEPortal.git**

3. Entrar a la carpeta raíz del proyecto: **cd Portal-NCIE-Pruebas**

Paso 3: Configuración de Base de Datos MySQL

1. Crear la base de datos:

```
sudo mysql
CREATE DATABASE u868517925_ncie;
```

2. Crear usuario y asignar permisos:

```
CREATE USER 'ncie_user'@'localhost' IDENTIFIED BY 'password_seguro';
GRANT ALL PRIVILEGES ON u868517925_ncie.* TO 'ncie_user'@'localhost';
FLUSH PRIVILEGES;
```

3. Importar la base de datos:

```
\q
(echo "SET FOREIGN_KEY_CHECKS=0;"; cat u868517925_ncie.sql; echo "SET
FOREIGN_KEY_CHECKS=1;") | mysql -u ncie_user -p u868517925_ncie
```

Paso 4: Configuración del Proyecto y Dependencias

1. Actualizar PHP, Módulos y Reiniciamos:

```
sudo apt install software-properties-common -y
sudo add-apt-repository ppa:ondrej/php -y
sudo apt update
sudo apt install php8.2 php8.2-cli php8.2-common php8.2-mysql php8.2-xml php8.2-mbstring
sudo apt install libapache2-mod-php8.2 -y
sudo a2dismod php8.1
sudo a2enmod php8.2
```

```
sudo systemctl restart apache2
```

2. Instalar las dependencias de PHP en modo producción:

```
composer install --optimize-autoloader --no-dev
```

3. Actualizar Node, e instalar las dependencias de Node.js: **curl -fsSL https://deb.nodesource.com/setup**
Ebash—sudo apt remove nodejs libnode-dev libnode72 -ysudo apt install -y nodejs

```
npm install
```

4. Compilar los assets del frontend para producción: **npm run build**
5. Duplicar el archivo .env.example y renombrarlo como .env: **cp .env.example .env**
6. El archivo .env deberá configurarse con los siguientes parámetros mínimos: Configuración de entorno para producción;

- a) APP_ENV=production
- b) APP_DEBUG=false
- c) APP_URL=http://IP_PUBLICA
- d) DB_CONNECTION=mysql
- e) DB_HOST=127.0.0.1
- f) DB_PORT=3306
- g) DB_DATABASE=nombre_base_datos
- h) DB_USERNAME=usuario
- i) DB_PASSWORD=contraseña
- j) LOG_CHANNEL=stack
- k) LOG_LEVEL=debug

Consideraciones:

- APP_DEBUG debe estar en false para evitar exposición de información sensible.
- APP_URL debe configurarse con la IP pública del servidor en ausencia de dominio.
- Las credenciales no deben ser compartidas ni versionadas.
- Para entorno de producción, se recomienda configurar LOG_LEVEL=error.

7. Generar la clave de encriptación: **php artisan key:generate**

8. **Aplicación de cambios de configuración:**

Después de modificar el archivo .env, se deben ejecutar los siguientes comandos:

php artisan config:clear

php artisan cache:clear

php artisan config:cache

9. Optimizar la configuración del framework:

php artisan route:cache

php artisan view:cache

Paso 5: Preparación de la Base de Datos y Almacenamiento

1. Crear el enlace simbólico para el almacenamiento público: **php artisan storage:link**

2. Ajustar los permisos de escritura críticos:

sudo chown -R www-data:www-data /var/www/ncieportal

sudo chmod -R 775 storage bootstrap/cache

Nota: El cambio de propietario debe aplicarse únicamente sobre el directorio del proyecto para evitar afectar otros archivos del sistema.

Paso 6: Puesta en Marcha Servidor Apache (Entorno Actual V1.0)

Dado que el entorno opera sobre un servidor web Apache, se debe configurar el Virtual Host apuntando al directorio público del proyecto Laravel.

1. Crear archivo de configuración del sitio en Apache: **sudo nano /etc/apache2/sites-available/ncie.com**

2. Definir el Virtual Host(ejemplo):

```
<VirtualHost *:80>
```

```
    ServerName 35.208.164.163
```

```
DocumentRoot /var/www/ncieportal/Portal-NCIE-Pruebas/public
```

```
<Directory /var/www/ncieportal/Portal-NCIE-Pruebas/public>  
    AllowOverride All  
    Require all granted  
</Directory>
```

```
ErrorLog ${APACHE_LOG_DIR}/ncie_error.log  
CustomLog ${APACHE_LOG_DIR}/ncie_access.log combined
```

```
</VirtualHost>
```

Nota: En ausencia de un dominio configurado por la institución, se deberá utilizar la dirección IP pública del servidor como valor de ServerName en la configuración del VirtualHost.

Adicionalmente, se recomienda definir el parámetro ServerName a nivel global en el archivo de configuración de Apache (/etc/apache2/apache2.conf) para evitar advertencias relacionadas con el nombre del servidor.

Posteriormente, este valor podrá ser actualizado al dominio institucional sin requerir cambios en la estructura del despliegue.

3. Habilitar el sitio y el módulo rewrite:

```
sudo a2ensite ncie.conf
```

```
sudo a2enmod rewrite
```

```
sudo a2dissite 000-default.conf
```

```
sudo systemctl restart apache2
```

Nota: Se deshabilita el sitio por defecto de Apache (000-default.conf) para evitar conflictos de enrutamiento y asegurar que el VirtualHost del proyecto sea el principal.

4. Verificar acceso desde navegador mediante la IP o dominio configurado.

Paso 7: Configurar Firewall

- `sudo ufw allow ssh`
- `sudo ufw allow http`

- **sudo ufw allow https**
- **sudo ufw allow 'Apache Full'**

```
sudo ufw --force enable
```

Nota: En entornos cloud como Google Cloud Platform, adicionalmente se deberán configurar reglas de firewall a nivel de red para permitir el acceso al puerto 80 (HTTP).

Paso 8: Endurecimiento básico de seguridad:

Para garantizar un entorno seguro en producción, se deberán aplicar las siguientes medidas:

1. Restricción de permisos del archivo `.env`: **chmod 600 /var/www/ncieportal/.env** Esto evita que otros usuarios del sistema puedan leer credenciales sensibles.
2. Deshabilitar acceso root por SSH: editar el archivo de configuración SSH:

```
sudo nano /etc/ssh/sshd_config
```

Modificar la siguiente línea:

PermitRootLogin no

Reiniciar el servicio: **sudo systemctl restart ssh**

3. Uso de usuario con privilegios controlados: El acceso al servidor deberá realizarse mediante un usuario estándar con permisos sudo, evitando el uso directo del usuario root.
4. Verificación de permisos en directorios críticos: Asegurar que únicamente el servidor web tenga permisos de escritura en:

storage/

bootstrap/cache

Paso 9: Integración con Servicios Externos

El sistema podrá integrarse con servicios de correo institucional mediante configuración SMTP en el archivo `.env`.

La configuración de DNS, cuentas de correo y servidores SMTP será responsabilidad de la institución.

Parámetros esperados:

```
MAIL_MAILER=  
MAIL_HOST=  
MAIL_PORT=  
MAIL_USERNAME=  
MAIL_PASSWORD=  
MAIL_ENCRYPTION=  
MAIL_FROM_ADDRESS=""  
MAIL_FROM_NAME=""
```

Estos valores deberán ser proporcionados por el área de infraestructura o TI.

6. Validación del Despliegue

Una vez configurado el servidor web, se deben realizar las siguientes verificaciones para asegurar el correcto funcionamiento del sistema:

6.1. Estado del servicio Apache

Se debe comprobar que el servicio Apache se encuentra activo mediante el siguiente comando:

```
systemctl status apache2
```

6.2. Accesibilidad local

Se valida que el servidor responde correctamente desde la misma máquina utilizando:

```
curl http://localhost
```

6.3. Acceso externo

Se debe acceder desde un navegador web utilizando la dirección IP pública del servidor:

```
http://IP_PUBLICA
```

6.4. Verificación de logs

Para monitoreo en tiempo real, se recomienda visualizar los logs del sistema mediante los siguientes comandos:

Logs de Laravel:

```
tail -f storage/logs/laravel.log
```

Logs de Apache:

```
sudo tail -f /var/log/apache2/error.log
```

Esto permite identificar errores en tiempo de ejecución y facilita la depuración del sistema.

6.5. Validación funcional

Se debe confirmar que el sistema cumple con los siguientes puntos:

1. Las rutas del sistema responden correctamente.
2. La conexión a la base de datos es exitosa.
3. Los recursos frontend (CSS y JavaScript) cargan adecuadamente.

7. Mantenimiento Operativo

Para garantizar la estabilidad y correcto funcionamiento del sistema, se deberán ejecutar periódicamente las siguientes tareas:

- **Limpieza de caché del sistema:** `php artisan optimize:clear`
- **Revisión de logs:** Archivo ubicado en: `storage/logs/laravel.log`
- **Verificación de estado de la base de datos:** Validar integridad, conexiones activas y consistencia de datos.
- **Monitoreo de espacio en disco:** Especial atención en directorios de almacenamiento y logs.
- **Validación de respaldos:** Confirmar que los backups se generen correctamente y sean recuperables.

8. Problemas Comunes y Soluciones

- **Error 500 (Internal Server Error):**

Causas comunes:

- Permisos incorrectos en `storage/` o `bootstrap/cache`

- Configuración incorrecta en el archivo **.env**
- Falta de clave de aplicación (APP_KEY)
- Caché desactualizado

Soluciones:

- Verificar permisos:

```
sudo chown -R www-data:www-data /var/www/ncieportal
sudo chmod -R 775 storage bootstrap/cache
```

- Generar clave:

```
php artisan key:generate
```

- Limpiar caché:

```
php artisan config:clear
php artisan cache:clear
php artisan config:cache
```

- Revisar logs:

```
storage/logs/laravel.log
```

■ **Error de conexión a base de datos:**

Verificar configuración en el archivo **.env**: **DB_HOST**, **DB_DATABASE**, **DB_USERNAME**, **DB_PASSWORD**.

■ **Fallo en dependencias de Node.js:**

Eliminar carpeta **node_modules/** y ejecutar:

```
npm install
```

■ **Problemas de permisos:**

Validar permisos en: **storage/ y bootstrap/cache.**

- **Recursos frontend no cargan:**

Ejecutar:

```
npm run build
```

9. Plan de Rollback y Recuperación ante Desastres

En caso de fallo crítico durante el despliegue o la operación, se ejecutarán los siguientes planes de contingencia para regresar a un estado estable.

9.1. Fallos durante el Despliegue Inicial (Rollback por instalación)

- **Detener Servicios:** Interrumpir **php artisan serve** y compilaciones de Vite.
- **Reversión de Base de Datos:** Si las migraciones corrompen datos, correr **php artisan migrate:reset** para limpiar tablas, o ingresar a MySQL y ejecutar un **DROP DATABASE** seguido de la recreación limpia.
- **Limpieza de Cachés:** Ejecutar de inmediato: **php artisan cache:clear**, **php artisan config:clear**, **php artisan route:clear**, **php artisan view:clear**
- **Reinicio de Dependencias:** Eliminar los directorios **vendor/** y **node_modules/** para forzar una instalación limpia mediante **composer install** y **npm install** en caso de conflicto de versiones.

9.2. Protocolos de Actuación por Incidente (Casos Críticos)

Caso A: Ciberataque o Acceso No Autorizado

1. **Aislamiento y Revocación:** Desconectar el servidor de la red institucional. Invalidar accesos SSH sospechosos y credenciales de MySQL.
2. **Restauración Limpia:** Clonar el repositorio desde cero en una instancia limpia (para evadir inyecciones en código).
3. **Recuperación de Datos:** Restaurar el último backup de BD previo al punto de infección.
4. **Expiración de Sesiones:** Ejecutar **php artisan key:generate** para obligar al cierre absoluto de todas las sesiones web activas o secuestradas.

Caso B: Fallo Físico del Servidor (Corte eléctrico, Falla de Disco)

1. **Ruteo de Emergencia:** Migrar el tráfico a un servidor físico o máquina virtual secundaria.
2. **Reinstalación Express:** Ejecutar los Pasos 1 al 4 del Procedimiento de Despliegue v1.0 en la nueva máquina.
3. **Restauración de Volumen:** Sobrescribir la carpeta **storage/app/public** y la Base de Datos con el último volcado guardado en el NAS institucional.

10. Política de Respaldos Cíclicos (Backups Automáticos)

Para sustentar estas recuperaciones, el servidor operará bajo rutinas de respaldo automatizadas:

- **Ejecución (Cron Job):** Programado diariamente a las 02:00 a.m. (franja de inactividad).
- **Alcance Técnico:** Vaciado integral de MySQL mediante:

```
mysqldump -u root -p u868517925_ncie > NCIE_backup.sql
```

y archivo comprimido **.tar.gz** del directorio **storage/app/public** (activos digitales y expedientes).

- **Retención Off-Site:** Rotación de 7 días locales, enviando una réplica inmediata mediante red hacia un disco externo o almacenamiento NAS institucional.

11. Buenas Prácticas de Producción

Para garantizar la estabilidad, seguridad y rendimiento del sistema en un entorno productivo, se recomienda aplicar las siguientes prácticas:

- **Deshabilitar modo debug en producción:** APP_DEBUG debe mantenerse en false para evitar exposición de información sensible.
- **Uso de HTTPS:** Se recomienda implementar certificados SSL (por ejemplo, Let's Encrypt) para asegurar la comunicación entre cliente y servidor.
- **Control de acceso:** Restringir acceso SSH mediante usuarios con privilegios limitados y evitar el uso del usuario root.

-
- **Separación de entornos:** No utilizar configuraciones de desarrollo en producción y mantener archivos .env diferenciados por entorno.
 - **Gestión de logs:** Se recomienda implementar mecanismos de rotación de logs para evitar saturación de almacenamiento y facilitar el monitoreo.
 - **Actualización del sistema:** Mantener actualizado el sistema operativo y dependencias mediante herramientas de gestión de paquetes.